CR YOUTH 2026 HACKATHON · SUBTEMA SUSTAINABLE DEVELOPMENT

RIVORA

PRD & SRS Prototipe

“See The Flood Before You See The Water.”

Sistem prediksi dan pencegahan genangan berbasis kondisi saluran drainase

Backend · **Dasbor Kelurahan** · **Aplikasi Pelapor**

Tim Abis Ini ke WDP · Telkom University

I Gede Ananda Bela Persada · Fatih Akram Wicaksono · Wildan Panji Maulana

Versi 1.0 · 8 November 2026

Acuan isi: Proposal Ide Hackathon CR YOUTH 2026 (RIVORA). Hal yang tidak tertulis di proposal ditandai [ASUMSI] atau [DEMO].

Sumber dokumen ini adalah file markdown di repo (`docs/backend/SRS.md`, `docs/dashboard/SRS.md`, `docs/mobile/SRS.md`). Bila berbeda, file di repo yang berlaku.

Daftar isi

DOKUMEN 1

Backend

Pemilik: B · Backend developer · `docs/backend/SRS.md`

1. Tujuan
2. Pengguna backend
3. Cakupan
4. Ukuran sukses (checkpoint `RENCANA_FINAL.md` bagian 8)
5. Arsitektur (Lampiran A)
6. Kebutuhan fungsional
7. Spesifikasi API
8. Model data (usulan untuk `db/schema.sql`)
9. Mesin skor risiko (3.3, Tabel 2)
10. Antarmuka modul AI (`backend/src/ai/`, dikerjakan C)
11. Notifikasi FCM
12. Kebutuhan non-fungsional
13. Uji terima (dasar Tabel 8 + `RENCANA_FINAL.md` bagian 10)
14. Urutan kerja B (Tabel 14)
15. Keputusan yang perlu disepakati (default berlaku bila tidak ada keberatan)

DOKUMEN 2

Dasbor Kelurahan

Pemilik: C · AI dan dasbor · `docs/dashboard/SRS.md`

1. Tujuan
2. Pengguna
3. Cakupan
4. Ukuran sukses (checkpoint `RENCANA_FINAL.md` bagian 8)
5. Tata letak
6. Kebutuhan fungsional
7. Data dari backend
8. Spesifikasi visual
9. Kebutuhan non-fungsional
10. Teknis
11. Uji terima
12. Urutan kerja C untuk dasbor (Tabel 14)
13. Keputusan yang perlu disepakati (default berlaku bila tidak ada keberatan)

DOKUMEN 3

Aplikasi Pelapor (Mobile)

Pemilik: A · Mobile developer · `docs/mobile/SRS.md`

1. Tujuan
2. Pengguna
3. Cakupan
4. Ukuran sukses
5. Alur layar
6. Kebutuhan fungsional
7. Endpoint yang dipakai
8. Kebutuhan non-fungsional
9. Uji terima
10. Urutan kerja A (Tabel 14)
11. Yang dibutuhkan dari anggota lain

Cara membaca

Siapa	Baca
Semua anggota	Bagian I (PRD) di tiap dokumen · Dokumen 1 bagian 7 (kontrak API) dan 9 (mesin skor)
B · Backend	Dokumen 1 penuh · Dokumen 3 bagian 7 dan 11 (yang dibutuhkan mobile)
C · AI dan dasbor	Dokumen 1 bagian 10 (modul AI) · Dokumen 2 penuh
A · Mobile	Dokumen 3 penuh · Dokumen 1 bagian 7.1–7.3, 7.9–7.11 dan 11 (FCM)

Penanda prioritas: P0 wajib jalan di checkpoint jam 10 · P1 setelah semua P0 hijau · P2 kalau sempat. **[ASUMSI]** angka/aturan yang belum ditetapkan proposal · **[DEMO]** alat bantu presentasi, wajib disebut ke juri.

PRD & SRS · Backend RIVORA

Pemilik: B · Backend developer (modul `backend/src/ai/` dikerjakan C, lihat bagian 10)
Versi: 1.0 · 8 November 2026 · **Status:** usulan, disepakati di Discord sebelum jam 1
Acuan: proposal 3.2, 3.3, Tabel 2, 3, 8, 12, 13, 14, Lampiran A dan C · `docs/RENCANA_FINAL.md` · `docs/API.md` · `docs/mobile/01-alur-lapor.md`

> `docs/API.md` tetap jadi kontrak resmi milik B. Dokumen ini menjelaskan **apa yang harus dibangun dan kenapa**, plus usulan detail kontrak. Bagian yang disetujui disalin B ke `API.md` dan `db/schema.sql`.

Penanda

Tanda	Arti
P0	Wajib jalan di checkpoint jam 10. Tanpa ini siklus demo putus
P1	Harus ada, dikerjakan setelah semua P0 hijau
P2	Kalau sempat. Boleh dilewati
[ASUMSI]	Angka/aturan yang belum ditetapkan proposal. Boleh diubah, sebut sebagai asumsi saat pitch
[DEMO]	Alat bantu presentasi, bukan fitur produk. Wajib disebut terus terang ke juri

---

Bagian I · PRD

1. Tujuan

Backend adalah satu-satunya pintu data untuk aplikasi pelapor (Flutter) dan dasbor kelurahan (React) (Tabel 12). Tugasnya menjalankan siklus **Lapor** → **Prediksi** → **Bertindak** → **Verifikasi** (3.2):

1. **Lapor:** menerima foto + GPS, meminta AI menilai sumbatan, menyimpan laporan.
2. **Prediksi:** menghitung skor risiko per RW dari hujan, sumbatan, dan kerentanan (3.3).
3. **Bertindak:** menyusun daftar prioritas saluran dan mengirim tugas ke petugas lewat FCM.
4. **Verifikasi:** menerima foto sesudah, meminta AI membandingkan, lalu memperbarui skor dan status.

2. Pengguna backend

Klien	Pengguna akhir	Yang dibutuhkan dari backend
Aplikasi pelapor	Warga, petugas kebersihan, siskamling	Kirim laporan, lihat hasil AI dan status, terima tugas, kirim foto sesudah
Dasbor web	Lurah, koordinator lapangan	Peta risiko RW, pin saluran, daftar prioritas, daftar petugas, buat tugas, pantau tugas
Sistem	–	Ambil prakiraan hujan berkala, hitung skor, panggil AI

3. Cakupan

Masuk (Tabel 3, fitur prototipe): lapor foto + GPS, analisis sumbatan AI, peta risiko, daftar prioritas sebelum hujan, penugasan dan notifikasi, verifikasi sebelum–sesudah.

Tidak masuk hari ini (Tabel 3 "tahap lanjut", 2.2): peringatan warga, mode kemarau, rekap & ekspor, integrasi LAPOR! dan Command Center, model YOLOv8, model gradient boosting, banyak kelurahan sekaligus.

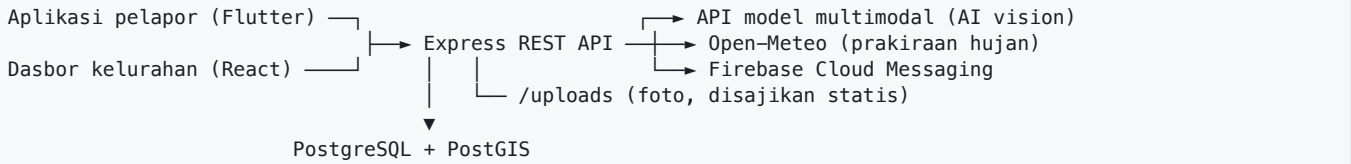
4. Ukuran sukses (checkpoint `RENCANA_FINAL.md` bagian 8)

Jam	Bukti yang harus bisa ditunjukkan
5	`curl` kirim foto ke `POST /reports` → hasil AI asli tersimpan → `GET /rw/risk` menunjukkan skor RW berubah
8	Dasbor memuat `GET /rw/risk`, `GET /saluran`, `GET /priorities` dengan data contoh + prakiraan Open-Meteo asli
10	Siklus penuh: HP lapor → dasbor tugaskan → HP terima FCM → HP kirim foto sesudah → skor RW turun di dasbor

---

Bagian II · SRS

5. Arsitektur (Lampiran A)



Struktur folder yang disarankan:

```
backend/
├── src/
│   ├── index.js            # app Express, CORS, static /uploads, error handler
│   ├── db.js               # pool pg
│   ├── auth.js             # login, middleware JWT (wajib / opsional / role)
│   ├── routes/             # reports.js, rw.js, saluran.js, priorities.js, tasks.js, devices.js, demo.js
│   ├── risk/score.js       # rumus skor (bagian 9), murni fungsi, mudah dites
│   ├── weather/openMeteo.js
│   ├── notify/fcm.js
│   └── ai/                 # milik C (bagian 10)
├── uploads/                # .gitignore, kecuali seed
└── package.json            # npm run dev | start | seed
```

6. Kebutuhan fungsional

6.1 Autentikasi

FR-BE-01 · Login (P1) · Tabel 12 "Autentikasi"

- `POST /auth/login` dengan akun dari data contoh: `lurah`, `petugas1`, `petugas2`, `warga`.
- Mengembalikan JWT (HS256, berlaku 24 jam **[ASUMSI]**) berisi `sub` (id pengguna) dan `role`.
- Password disimpan sebagai hash bcrypt. Password demo dibaca dari `SEED_PASSWORD` di `.env`, **tidak ditulis di repo**.
- Aturan akses:

Endpoint	Akses
`POST /reports`, `GET /reports/:id`	Token **opsional**. Tanpa token = warga anonim, `pelapor_id` kosong **[ASUMSI]** (mobile tahap 01 belum login)
`GET /rw/risk`, `GET /saluran*`, `GET /priorities`	Terbuka (data agregat) **[ASUMSI]** supaya peta bisa dikerjakan sebelum login jadi
`GET /petugas`, `POST /tasks`, `GET /tasks?kelurahan_id=`, `PUT /demo/hujan`	Token `lurah`
`GET /tasks?mine=1`, `PATCH /tasks/:id`, `POST /tasks/:id/verify`	Token `petugas` (tugas miliknya sendiri)
`POST /devices`	Token apa saja

Kriteria terima: password salah → 401. Token petugas memanggil `POST /tasks` → 403. Tanpa token memanggil `POST /reports` → 201.

6.2 Laporan dan analisis AI

FR-BE-02 · Terima laporan (P0) · 3.2 langkah 1, Tabel 3

- `POST /reports` multipart: `foto`, `lat`, `lng`, `captured_at`, `jenis_saluran` (persis seperti yang dikirim mobile, `docs/mobile/01-alur-lapor.md`).
- Urutan proses:
  1. Validasi input (FR-BE-03).
  2. Simpan foto ke `uploads/laporan/<uuid>.jpg`, buat `foto_url` absolut.
  3. Tentukan RW dan saluran (FR-BE-04).
  4. Panggil `analyzeReport()` dari modul AI (FR-BE-05).
  5. Tentukan status: `cek_manual` bila kelas `tidak_terlihat` atau AI gagal; `ditugaskan` bila saluran sedang punya tugas aktif; selain itu `dikirim`.
  6. Simpan baris `laporan`, kembalikan **201** dengan bentuk respons bagian 7.2.
- Skor RW **tidak disimpan**: dihitung saat dibaca (bagian 9), jadi otomatis "diperbarui" begitu laporan masuk.

Kriteria terima: respons bisa di-parse `Report.fromJson` mobile tanpa perubahan kode mobile. Laporan baru langsung memengaruhi `GET /rw/risk` berikutnya.

FR-BE-03 · Validasi input (P1) · Tabel 8 "laporan palsu"

- `foto` wajib, `image/jpeg` atau `image/png`, maksimal 10 MB **[ASUMSI]** (batas multer).
- `lat` −90..90, `lng` −180..180, `jenis_saluran` salah satu enum.
- `captured_at` ISO 8601, tidak lebih dari 5 menit di masa depan dan tidak lebih tua dari 60 menit **[ASUMSI]**. Ini lapisan server untuk aturan "foto langsung dari kamera" (3.2). Data contoh dimasukkan lewat seed, jadi tidak terkena aturan ini.
- Gagal validasi → **400** `{ "error": "<pesan dalam Bahasa Indonesia>" }`. Mobile menampilkan pesan ini apa adanya.

FR-BE-04 · Penentuan RW dan penggabungan saluran (P0 untuk RW, P1 untuk penggabungan) · Tabel 8 "laporan berdekatan digabung otomatis"

- Cari saluran terdekat dalam radius **15 m [ASUMSI]**: `ST_DWithin(saluran.geom, titik, 15)` urut `ST_Distance`. Ketemu → laporan ditempel ke saluran itu, respons `digabung: true`.
- Tidak ketemu → buat `saluran` baru di titik itu dengan `jenis` dari pelapor dan `rw_id` dari `ST_Covers(rw.geom, titik)`.
- Titik di luar semua poligon RW → `rw_id = NULL`, tidak masuk skor RW mana pun. Pengecualian demo: FR-BE-21.

FR-BE-05 · Analisis sumbatan AI (P0) · 3.3, Lampiran C

- Backend memanggil `analyzeReport({ foto, mimeType, jenisSaluran })` dari `src/ai` (antarmuka bagian 10). Backend **tidak** memanggil API model secara langsung.
- Hasil disimpan utuh di `laporan.ai_json`, dan field `kelas`, `skor_sumbatan`, `keyakinan`, `ai_sumber` disalin ke kolom sendiri.
- Bila `jenis_saluran` dari AI berbeda dari pilihan pelapor, pilihan pelapor yang dipakai untuk `saluran.jenis` **[ASUMSI]**. Keduanya tetap tersimpan.

FR-BE-06 · AI gagal tidak menggagalkan laporan (P0) · Tabel 8 "kelas tidak terlihat memicu pengecekan manual"

- AI timeout, error, atau JSON tetap tidak valid setelah 1 kali ulang → laporan **tetap disimpan** dengan `status: cek_manual`, `ai_sumber: gagal`, dan `ai` berisi:

```
{ "saluran_terlihat": false, "kelas_sumbatan": "tidak_terlihat", "skor_sumbatan": null,
  "penyebab": [], "keyakinan": "rendah", "catatan": "Analisis AI gagal, perlu dicek manual" }
```

- Mobile sudah menangani bentuk ini (banner "Perlu dicek manual").

FR-BE-07 · Status laporan (P1) · Tabel 8 "status sudah ditangani beserta foto dikirim kembali ke pelapor"

- `GET /reports/:id` → data laporan + `tugas` (status, nama petugas) + `bukti` (foto sesudah, kelas sesudah, waktu selesai) bila sudah ditangani.
- Mobile menyimpan id laporan di HP, jadi warga anonim tetap bisa melihat statusnya.
- **P2:** `GET /reports?mine=1` (butuh token) untuk riwayat laporan pelapor yang login.

6.3 Prediksi

FR-BE-08 · Prakiraan hujan (P0) · Tabel 2 "Hujan", Tabel 12

- Ambil dari `OPEN_METEO_URL` dengan `latitude` / `longitude` = centroid kelurahan, `hourly=precipitation`, `forecast_days=2`, `timezone=UTC`.
- `total_24j_mm` = jumlah `precipitation` dari jam sekarang sampai 24 jam ke depan.
- Simpan di `prakiraan_hujan`, pakai ulang selama **30 menit [ASUMSI]**.
- Open-Meteo gagal → pakai cache terakhir. Belum ada cache → `total_24j_mm = 0` dengan `hujan_sumber: "tidak_tersedia"`. Endpoint lain tidak boleh ikut error.
- BMKG: tahap lanjut, tidak dikerjakan hari ini.

FR-BE-09 · Skor risiko per RW (P0) · 3.3, Tabel 2

- `GET /rw/risk?kelurahan_id=` → GeoJSON `FeatureCollection` poligon RW. Tiap fitur membawa `skor`, `level`, rincian tiga komponen beserta bobotnya, dan kalimat `alasan` (bagian 7.4).
- Rumus dan normalisasi: bagian 9.
- RW tanpa satu pun laporan yang terlihat → `S = 0` dengan `komponen.sumbatan.data_cukup: false`, supaya dasbor bisa menandai "belum ada data" dan RW itu tidak terlihat aman padahal belum pernah dicek (3.3, transparansi).

Kriteria terima: dengan data contoh, minimal ada 1 RW di tiap level saat simulasi hujan 80 mm (FR-BE-20). Perhitungan cocok dengan contoh hitung bagian 9.4.

FR-BE-10 · Daftar saluran untuk peta (P0) · Tabel 3 "titik saluran bermasalah"

- `GET /saluran?kelurahan_id=` → GeoJSON titik saluran + kondisi terkini + status tugas aktif.

FR-BE-11 · Detail saluran (P1)

- `GET /saluran/:id` → info saluran, riwayat laporan (terbaru dulu), dan riwayat tugas. Dipakai panel detail dan tampilan sebelum–sesudah di dasbor.

6.4 Bertindak

FR-BE-12 · Daftar prioritas sebelum hujan (P0) · 3.2 langkah 3, Tabel 3

- `GET /priorities?kelurahan_id=` → saluran dengan kondisi terkini `ringan` / `sedang` / `berat`, diurutkan menurun berdasarkan `skor_prioritas` (bagian 9.3).
- Saluran yang kondisi terkininya `bersih` tidak masuk daftar **[ASUMSI]**.
- Saluran yang laporan terbarunya `tidak_terlihat` masuk daftar terpisah `cek_manual`.
- `meta.hujan_lebat_diprediksi = total_24j_mm ≥ 50` (ambang hujan lebat BMKG). Daftar tetap dikirim walau flag `false`; dasbor yang memberi konteks.

FR-BE-13 · Daftar petugas (P0)

- `GET /petugas` → petugas di kelurahan lurah yang login + jumlah tugas aktif masing-masing. Untuk pilihan di dialog Tugaskan.

FR-BE-14 · Buat tugas + notifikasi (P0) · 3.2 langkah 3, Tabel 3 "Penugasan dan notifikasi"

- `POST /tasks` `{ saluran_id, petugas_id, catatan? }`.
- Satu saluran hanya boleh punya satu tugas yang belum selesai → kedua kalinya **409** `{ "error": "Saluran ini sudah punya tugas aktif" }` (dijaga unique index parsial, bagian 8).
- `laporan_id` = laporan terbaru saluran itu. Semua laporan saluran itu yang berstatus `dikirim` → `ditugaskan`.
- Kirim FCM ke `fcm_token` petugas (bagian 11). FCM gagal **tidak** membatalkan tugas. Respons memberi tahu hasilnya: `notifikasi: "terkirim" | "gagal" | "tanpa_token"`.

FR-BE-15 · Daftar tugas (P0)

- `GET /tasks?mine=1` (petugas) → tugas milik sendiri yang belum selesai lebih dulu. Ini juga cadangan kalau FCM tidak sampai: mobile cukup menarik ulang daftar.
- `GET /tasks?kelurahan_id=` (lurah) → semua tugas di kelurahan, untuk tab Tugas di dasbor.

FR-BE-16 · Simpan token FCM (P0)

- `POST /devices` `{ fcm_token }` → simpan ke `pengguna.fcm_token` milik pemilik token JWT. **204**.

FR-BE-23 · Tandai dikerjakan (P2)

- `PATCH /tasks/:id` `{ "status": "dikerjakan" }` oleh petugas pemilik tugas.

6.5 Verifikasi

FR-BE-17 · Verifikasi sebelum–sesudah (P0) · 3.2 langkah 4, Tabel 3

- `POST /tasks/:id/verify` multipart: `foto`, `lat`, `lng`, `captured_at` (sama dengan laporan).
- Pemeriksaan: tugas milik petugas yang login; status belum `selesai`; posisi dalam **30 m [ASUMSI]** dari saluran (dilewati bila laporan memakai lokasi demo, FR-BE-21); `captured_at` seperti FR-BE-03.
- Panggil `verifyCleaning({ fotoSebelum, fotoSesudah })`. `fotoSebelum` = foto laporan di `tugas.laporan_id`.
- `sudah_bersih: true`:
  - `tugas.status = selesai`, simpan `foto_sesudah_url`, `ai_verifikasi_json`, `selesai_at`.
  - Sisipkan baris `laporan` baru dengan `sumber: verifikasi`, `pelapor_id` = petugas, `kelas` / `skor_sumbatan` dari hasil verifikasi, `status: ditangani`. Baris ini jadi kondisi terkini saluran, sehingga skor RW turun, sekaligus menambah dataset drainase (3.2 langkah 4).
  - Semua laporan saluran itu yang berstatus `dikirim` / `ditugaskan` → `ditangani`.
- `sudah_bersih: false` → `tugas.status = dikerjakan`, tidak ada perubahan lain, respons tetap **200** dengan catatan AI supaya petugas bisa membersihkan lagi lalu foto ulang.
- AI gagal → **200** dengan `verifikasi: null` dan `pesan: "Verifikasi AI gagal, coba foto ulang"`. Tugas tetap `dikerjakan`.

Kriteria terima: setelah verifikasi berhasil, `GET /rw/risk` menunjukkan `skor` RW lebih kecil dan `GET /reports/:id` (laporan awal) berstatus `ditangani` dengan `bukti` terisi.

6.6 Data contoh dan pengaman demo

FR-BE-18 · Data contoh (P0) · Tabel 14 jam 0–1, `RENCANA_FINAL.md` bagian 9

- `npm run seed` mengosongkan lalu mengisi ulang database:
  - 1 kelurahan (salah satu kelurahan di Gedebage atau Kopo) beserta centroid.
  - 10–15 poligon RW dari geojson.io, dengan `elevasi_m`, `elevasi_relatif`, `jarak_genangan_m`, `kerentanan` yang sudah dihitung (bagian 9.2).
  - Titik genangan langganan dari sumber [3] proposal.
  - Sekitar 20 saluran dan 30 laporan dummy (foto di `db/seed/foto/`) dengan kelas campuran, termasuk 2 `tidak_terlihat`. `captured_at` tersebar dalam 7 hari terakhir.
  - Akun `lurah`, `petugas1`, `petugas2`, `warga`.
- Seed harus deterministik: menjalankannya dua kali menghasilkan data yang sama. Ini penting untuk rekam video dan latihan pitch.
- Siapkan **satu RW demo** dengan 1–2 saluran saja, supaya penurunan skor setelah verifikasi terlihat jelas di peta.

FR-BE-19 · Mode demo AI (P1) [DEMO] · `RENCANA_FINAL.md` bagian 12

- `DEMO_MODE=1`: bila model AI timeout atau gagal, modul AI mengembalikan hasil contoh bergiliran (berat → sedang → bersih, mirip `MockApiClient` mobile) dengan `ai_sumber: demo`.
- `ai_sumber` ikut dikirim di respons dan disimpan, supaya tim selalu tahu mana hasil model asli dan mana hasil contoh.

FR-BE-20 · Simulasi hujan (P1) [DEMO] · `RENCANA_FINAL.md` bagian 12

- Hanya aktif bila `DEMO_MODE=1`. `PUT /demo/hujan` `{ "mm": 80 }` atau `{ "mm": null }` untuk kembali ke Open-Meteo. Nilai disimpan di memori proses, hilang saat server restart.

- Saat aktif, semua perhitungan (`/rw/risk`, `/priorities`, verifikasi) memakai nilai ini dan `meta.hujan_sumber = "simulasi_demo"`. Dasbor wajib menampilkan label simulasi.
- Alasan dibuat global (bukan query param): seluruh sistem menceritakan angka yang sama saat demo.

FR-BE-21 · Lokasi demo (P1) [DEMO]

- Masalah: lomba daring, jadi foto live diambil di lokasi tim, bukan di kelurahan contoh. Tanpa penanganan, laporan live jatuh di luar semua RW dan tidak tampil di skor.
- `DEMO_SALURAN_ID=<id saluran di RW demo>`: laporan yang titiknya di luar semua RW ditempel ke saluran ini. `laporan.geom` tetap menyimpan posisi asli dan `laporan.lokasi_demo = true`.
- Tanpa env ini → perilaku normal FR-BE-04.
- **Wajib disebut saat demo:** "laporan live dipetakan ke saluran contoh karena kami tidak sedang berada di kelurahan pilot."

FR-BE-22 · Health check (P1)

- `GET /health` → `{ "ok": true, "demo_mode": true, "ai_siap": true, "db": "ok" }`. Dasbor memakai `demo_mode` untuk menampilkan atau menyembunyikan slider simulasi.

7. Spesifikasi API

Aturan umum:

- JSON memakai `snake_case`. Waktu ISO 8601 UTC (`2026-11-08T02:30:00.000Z`). Id berupa angka.
- Koordinat di body JSON memakai `lat` dan `lng` terpisah. Di GeoJSON, urutannya **`[lng, lat]`** sesuai standar.
- Semua error: `{ "error": "pesan singkat Bahasa Indonesia" }` dengan status 400/401/403/404/409/500. Tanpa stack trace.
- `foto_url` selalu URL absolut (`https://<host>/uploads/...`).

7.1 `POST /auth/login`

```
// request
{ "username": "lurah", "password": "••••" }
// 200
{ "token": "eyJ...", "role": "lurah", "pengguna": { "id": 1, "nama": "Lurah Contoh", "kelurahan_id": 1 } }
```

7.2 `POST /reports` → 201

Field yang sudah dibaca mobile ditulis lebih dulu. Field sesudah `ai` adalah tambahan; mobile mengabaikannya tanpa error.

```
{
  "id": 42,
  "status": "dikirim",
  "jenis_saluran": "terbuka",
  "lat": -6.940123,
  "lng": 107.690456,
  "captured_at": "2026-11-08T02:30:00.000Z",
  "foto_url": "https://api.contoh/uploads/laporan/7f3c.jpg",
  "ai": {
    "jenis_saluran": "terbuka",
    "saluran_terlihat": true,
    "kelas_sumbatan": "berat",
    "skor_sumbatan": 0.85,
    "penyebab": ["plastik", "sedimen"],
    "keyakinan": "tinggi",
    "catatan": "Sampah plastik menutupi sebagian besar lebar saluran"
  },
  "ai_sumber": "model",
  "saluran_id": 7,
  "digabung": false,
  "rw_id": 5,
  "rw_nama": "RW 05",
  "lokasi_demo": false
}
```

7.3 `GET /reports/:id` → 200

Sama dengan 7.2, ditambah:

```
{
  "tugas": { "id": 12, "status": "selesai", "petugas_nama": "Petugas 1" },
  "bukti": {
    "foto_sesudah_url": "https://api.contoh/uploads/verifikasi/91aa.jpg",
    "kelas_sesudah": "bersih",
    "selesai_at": "2026-11-08T05:10:00.000Z"
  }
}
```

`tugas` dan `bukti` bernilai `null` bila belum ada.

7.4 `GET /rw/risk?kelurahan_id=1` → 200

```
{
  "type": "FeatureCollection",
  "meta": {
    "kelurahan": { "id": 1, "nama": "Kelurahan Contoh" },
    "hujan_24j_mm": 80,
    "hujan_sumber": "open_meteo",
    "hujan_diambil_at": "2026-11-08T03:00:00.000Z",
    "hujan_lebat_diprediksi": true,
    "bobot": { "hujan": 0.4, "sumbatan": 0.4, "kerentanan": 0.2 },
    "dihitung_at": "2026-11-08T03:05:12.000Z"
  },
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "Polygon", "coordinates": [[[107.69, -6.94], [107.692, -6.94], [107.692, -6.942],
[107.69, -6.942], [107.69, -6.94]]] },
      "properties": {
        "rw_id": 5,
        "nama": "RW 05",
        "skor": 0.80,
        "level": "risiko_tinggi",
        "komponen": {
          "hujan":      { "nilai": 0.80, "bobot": 0.4, "kontribusi": 0.32, "mm": 80 },
          "sumbatan":   { "nilai": 0.85, "bobot": 0.4, "kontribusi": 0.34, "jumlah_saluran": 1,
"jumlah_cek_manual": 0, "data_cukup": true },
          "kerentanan": { "nilai": 0.70, "bobot": 0.2, "kontribusi": 0.14, "elevasi_relatif": 0.9,
"jarak_genangan_m": 500, "genangan_terdekat": "Pasar Induk Gedebage" }
        },
        "alasan": "Risiko tinggi: prakiraan hujan 80 mm dalam 24 jam, 1 dari 1 saluran tersumbat sedang–berat,
lokasi rendah dan 500 m dari titik genangan Pasar Induk Gedebage."
      }
    }
  ]
}
```

`hujan_sumber`: `open_meteo` · `cache` · `tidak_tersedia` · `simulasi_demo`.

7.5 `GET /saluran?kelurahan_id=1` → 200

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": { "type": "Point", "coordinates": [107.690456, -6.940123] },
      "properties": {
        "saluran_id": 7,
        "jenis": "terbuka",
        "rw_id": 5,
        "rw_nama": "RW 05",
        "kondisi": {
          "laporan_id": 42,
          "kelas": "berat",
          "skor_sumbatan": 0.85,
          "keyakinan": "tinggi",
          "foto_url": "https://api.contoh/uploads/laporan/7f3c.jpg",
          "captured_at": "2026-11-08T02:30:00.000Z"
        },
        "perlu_cek_manual": false,
        "tugas_aktif": { "id": 12, "status": "baru", "petugas_nama": "Petugas 1" }
      }
    }
  ]
}
```

`kondisi` = laporan terbaru yang terlihat (bagian 9.1), `null` bila belum ada. `tugas_aktif` bernilai `null` bila tidak ada tugas yang belum selesai.

7.6 `GET /saluran/:id` → 200

```
{
  "saluran_id": 7, "jenis": "terbuka", "rw_id": 5, "rw_nama": "RW 05", "lat": -6.940123, "lng": 107.690456,
  "riwayat": [
    { "laporan_id": 50, "sumber": "verifikasi", "kelas": "bersih", "skor_sumbatan": 0.08, "keyakinan": "tinggi",
      "foto_url": "…", "captured_at": "…", "status": "ditangani", "ai_sumber": "model", "lokasi_demo": false,
"catatan": "…" },
    { "laporan_id": 42, "sumber": "laporan", "kelas": "berat", "skor_sumbatan": 0.85, "keyakinan": "tinggi",
      "foto_url": "…", "captured_at": "…", "status": "ditangani", "ai_sumber": "model", "lokasi_demo": false,
"catatan": "…" }
  ],
  "tugas": [ { "…": "bentuk sama dengan 7.9" } ]
}
```

7.7 `GET /priorities?kelurahan_id=1` → 200

```
{
  "meta": { "hujan_24j_mm": 80, "hujan_sumber": "open_meteo", "hujan_lebat_diprediksi": true, "dihitung_at": "…"
},
  "prioritas": [
    {
      "peringkat": 1,
      "saluran_id": 7, "jenis": "terbuka", "rw_id": 5, "rw_nama": "RW 05",
      "lat": -6.940123, "lng": 107.690456,
      "kelas": "berat", "skor_sumbatan": 0.85, "keyakinan": "tinggi",
      "foto_url": "…", "captured_at": "…",
      "skor_prioritas": 0.80,
      "komponen": { "hujan": 0.80, "sumbatan": 0.85, "kerentanan": 0.70 },
      "tugas_aktif": null
    }
  ],
  "cek_manual": [
    { "saluran_id": 9, "jenis": "gorong_gorong", "rw_nama": "RW 02", "foto_url": "…", "captured_at": "…",
      "catatan": "Saluran tertutup beton, kondisi di dalam tidak terlihat" }
  ]
}
```

7.8 `GET /petugas` → 200

```
[ { "id": 2, "nama": "Petugas 1", "tugas_aktif": 1 }, { "id": 3, "nama": "Petugas 2", "tugas_aktif": 0 } ]
```

7.9 `POST /tasks` → 201 · `GET /tasks` → 200 (array)

```
// request POST
{ "saluran_id": 7, "petugas_id": 2, "catatan": "Prioritas sebelum hujan sore" }
// objek tugas
{
  "id": 12,
  "status": "baru",
  "saluran": { "id": 7, "jenis": "terbuka", "rw_nama": "RW 05", "lat": -6.940123, "lng": 107.690456 },
  "petugas": { "id": 2, "nama": "Petugas 1" },
  "laporan_id": 42,
  "foto_sebelum_url": "…", "kelas_sebelum": "berat", "skor_sebelum": 0.85,
  "foto_sesudah_url": null,
  "verifikasi": null,
  "catatan": "Prioritas sebelum hujan sore",
  "dibuat_at": "…", "selesai_at": null
}
// respons POST membungkus objek di atas:
{ "tugas": { "…": "…" }, "notifikasi": "terkirim" }
```

7.10 `POST /tasks/:id/verify` → 200

```
{
  "tugas": { "…": "objek tugas 7.9, status sudah diperbarui" },
  "verifikasi": {
    "sudah_bersih": true,
    "kelas_sesudah": "bersih",
    "skor_sumbatan_sesudah": 0.08,
    "keyakinan": "tinggi",
    "catatan": "Sampah plastik sudah terangkat, aliran terlihat lancar"
  },
  "ai_sumber": "model"
}
```

7.11 Lainnya

Endpoint	Request	Respons
`POST /devices`	`{ "fcm_token": "…" }`	204
`PATCH /tasks/:id` (P2)	`{ "status": "dikerjakan" }`	200 objek tugas
`PUT /demo/hujan` [DEMO]	`{ "mm": 80 }` / `{ "mm": null }`	200 `{ "mm": 80 }`. 404 bila `DEMO_MODE` mati
`GET /health` (P1)	–	`{ "ok": true, "demo_mode": true, "ai_siap": true, "db": "ok" }`

8. Model data (usulan untuk `db/schema.sql`)

Perubahan dari draft di `RENCANA_FINAL.md` bagian 6: tambah `username` / `password_hash`, `laporan.sumber`, `laporan.ai_sumber`, `laporan.lokasi_demo`, `tugas.laporan_verifikasi_id`, dan unique index tugas aktif. Enum memakai `CHECK` (lebih mudah diubah saat hackathon dibanding `CREATE TYPE`).

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE kelurahan (
  id        SERIAL PRIMARY KEY,
  nama      TEXT NOT NULL,
  kecamatan TEXT,
  centroid  GEOGRAPHY(POINT, 4326) NOT NULL
);

CREATE TABLE pengguna (
  id            SERIAL PRIMARY KEY,
  username      TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  nama          TEXT NOT NULL,
  role          TEXT NOT NULL CHECK (role IN ('lurah', 'petugas', 'warga')),
  kelurahan_id  INT REFERENCES kelurahan(id),
  fcm_token     TEXT
);

CREATE TABLE rw (
  id               SERIAL PRIMARY KEY,
  kelurahan_id     INT NOT NULL REFERENCES kelurahan(id),
  nama             TEXT NOT NULL,
  geom             GEOGRAPHY(POLYGON, 4326) NOT NULL,
  elevasi_m        NUMERIC,                -- DEMNAS di centroid RW
  elevasi_relatif  NUMERIC,                -- 0..1, 1 = paling rendah di kelurahan
  jarak_genangan_m NUMERIC,                -- ke titik genangan langganan terdekat
  kerentanan       NUMERIC NOT NULL CHECK (kerentanan BETWEEN 0 AND 1)
);
CREATE INDEX rw_geom_idx ON rw USING GIST (geom);

CREATE TABLE titik_genangan (
  id     SERIAL PRIMARY KEY,
  nama   TEXT NOT NULL,
  geom   GEOGRAPHY(POINT, 4326) NOT NULL,
  sumber TEXT
);

CREATE TABLE saluran (
  id        SERIAL PRIMARY KEY,
  rw_id     INT REFERENCES rw(id),          -- NULL = di luar wilayah
  jenis     TEXT NOT NULL CHECK (jenis IN ('terbuka', 'gorong_gorong', 'inlet_jalan')),
  geom      GEOGRAPHY(POINT, 4326) NOT NULL,
  dibuat_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX saluran_geom_idx ON saluran USING GIST (geom);

CREATE TABLE laporan (
  id            SERIAL PRIMARY KEY,
  saluran_id    INT NOT NULL REFERENCES saluran(id),
  pelapor_id    INT REFERENCES pengguna(id), -- NULL = warga anonim
  sumber        TEXT NOT NULL DEFAULT 'laporan' CHECK (sumber IN ('laporan', 'verifikasi')),
  jenis_saluran TEXT NOT NULL CHECK (jenis_saluran IN ('terbuka', 'gorong_gorong', 'inlet_jalan')),
  foto_url      TEXT NOT NULL,
  geom          GEOGRAPHY(POINT, 4326) NOT NULL, -- posisi asli dari HP
  lokasi_demo   BOOLEAN NOT NULL DEFAULT false,
  captured_at   TIMESTAMPTZ NOT NULL,
  ai_json       JSONB,
  ai_sumber     TEXT NOT NULL CHECK (ai_sumber IN ('model', 'demo', 'gagal')),
  kelas         TEXT NOT NULL CHECK (kelas IN ('bersih', 'ringan', 'sedang', 'berat', 'tidak_terlihat')),
  skor_sumbatan NUMERIC CHECK (skor_sumbatan BETWEEN 0 AND 1),
  keyakinan     TEXT NOT NULL CHECK (keyakinan IN ('rendah', 'sedang', 'tinggi')),
  status        TEXT NOT NULL CHECK (status IN ('dikirim', 'ditugaskan', 'ditangani', 'cek_manual')),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX laporan_saluran_waktu_idx ON laporan (saluran_id, captured_at DESC);

CREATE TABLE tugas (
  id                    SERIAL PRIMARY KEY,
  saluran_id            INT NOT NULL REFERENCES saluran(id),
  laporan_id            INT NOT NULL REFERENCES laporan(id),
  petugas_id            INT NOT NULL REFERENCES pengguna(id),
  ditugaskan_oleh       INT REFERENCES pengguna(id),
  status                TEXT NOT NULL DEFAULT 'baru' CHECK (status IN ('baru', 'dikerjakan', 'selesai')),
```

```
  catatan               TEXT,
  foto_sesudah_url      TEXT,
  ai_verifikasi_json    JSONB,
  laporan_verifikasi_id INT REFERENCES laporan(id),
  dibuat_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  selesai_at            TIMESTAMPTZ
);
-- Satu saluran hanya boleh punya satu tugas yang belum selesai (FR-BE-14)
CREATE UNIQUE INDEX tugas_aktif_per_saluran ON tugas (saluran_id) WHERE status <> 'selesai';

CREATE TABLE prakiraan_hujan (
  id           SERIAL PRIMARY KEY,
  kelurahan_id INT NOT NULL REFERENCES kelurahan(id),
  fetched_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  total_24j_mm NUMERIC NOT NULL,
  hourly_json  JSONB NOT NULL
);
```

Catatan PostGIS:

- Kolom `GEOGRAPHY` membuat `ST_DWithin` / `ST_Distance` langsung dalam meter.
- Membuat titik dari input: `ST_SetSRID(ST_MakePoint($lng, $lat), 4326)::geography`. **Urutannya lng dulu.**
- GeoJSON keluar: `ST_AsGeoJSON(geom)::json`. Poligon dari geojson.io masuk: `ST_GeomFromGeoJSON($1)::geography`.
- Cek titik di dalam RW: `ST_Covers(rw.geom, titik)`.

9. Mesin skor risiko (3.3, Tabel 2)

```
Skor Risiko RW = 0,4 × H + 0,4 × S + 0,2 × K        (H, S, K dinormalisasi 0–1)
```

Bobot = asumsi awal proposal, dikalibrasi saat pilot. Taruh di satu konstanta `BOBOT` di `src/risk/score.js`.

9.1 Komponen

Komponen	Rumus
H · Hujan	`min(total_24j_mm / 100, 1)` **[ASUMSI: 100 mm = batas atas hujan lebat BMKG]**
S · Sumbatan	Untuk tiap saluran di RW, ambil **kondisi terkini** = laporan terbaru dengan `kelas ≠ tidak_terlihat` dan `ai_sumber ≠ gagal`. `S = Σ(w × skor_sumbatan) / Σw`, dengan `w` = 1,0 (keyakinan tinggi) / 0,6 (sedang) / 0,3 (rendah) **[ASUMSI]**. Tidak ada data → `S = 0`, `data_cukup = false`
K · Kerentanan	Dihitung saat seed, disimpan di `rw.kerentanan` (9.2)

Query kondisi terkini:

```
SELECT DISTINCT ON (l.saluran_id)
       l.saluran_id, l.kelas, l.skor_sumbatan, l.keyakinan, l.foto_url, l.captured_at, l.id AS laporan_id
FROM laporan l
JOIN saluran s ON s.id = l.saluran_id
JOIN rw ON rw.id = s.rw_id
WHERE rw.kelurahan_id = $1
  AND l.kelas <> 'tidak_terlihat'
  AND l.ai_sumber <> 'gagal'
ORDER BY l.saluran_id, l.captured_at DESC, l.id DESC;
```

Saluran **perlu cek manual** = laporan terbarunya (tanpa filter di atas) berkelas `tidak_terlihat` atau `ai_sumber = gagal`.

9.2 Kerentanan (dihitung di skrip seed)

```
elevasi_relatif = (elev_max − elev_rw) / (elev_max − elev_min)   dalam satu kelurahan, 1 = paling rendah
kedekatan       = 1 − min(jarak_genangan_m / 1000, 1)            [ASUMSI: di atas 1 km dianggap jauh]
K               = 0,5 × elevasi_relatif + 0,5 × kedekatan        [ASUMSI bobot 0,5/0,5]
```

`elev_rw` dari DEMNAS di centroid RW (cadangan: Open-Meteo Elevation API, sebut terus terang saat pitch). `jarak_genangan_m` = `ST_Distance(centroid RW, titik_genangan terdekat)`.

9.3 Level dan prioritas

- **Level [ASUMSI]:** `aman` < 0,40 · `waspada` 0,40–0,69 · `risiko_tinggi` ≥ 0,70.
- **Skor prioritas saluran [ASUMSI, turunan rumus]:** `0,4 × H + 0,4 × skor_sumbatan_saluran + 0,2 × K_rw`. Seri → laporan lebih baru di atas.
- **Hujan lebat diprediksi:** `total_24j_mm ≥ 50`.
- Tanpa hujan (H = 0), skor maksimal 0,60, jadi tidak ada RW berlevel risiko tinggi. Ini sengaja: risiko tinggi butuh hujan.

9.4 Contoh hitung (pakai untuk unit test)

RW 05: 1 saluran, kerentanan K = 0,70, simulasi hujan 80 mm → H = 0,80.

Kondisi	S	Skor	Level
Laporan "berat" 0,85, keyakinan tinggi	0,85	0,32 + 0,34 + 0,14 = **0,80**	risiko_tinggi
Setelah verifikasi "bersih" 0,08	0,08	0,32 + 0,032 + 0,14 = **0,49**	waspada
Tanpa hujan, sebelum dibersihkan	0,85	0 + 0,34 + 0,14 = **0,48**	waspada

9.5 Kalimat `alasan`

Template:

```
{Level}: prakiraan hujan {mm} mm dalam 24 jam, {n_sedang_berat} dari {n_saluran} saluran tersumbat sedang–berat,
{lokasi rendah | lokasi sedang | lokasi relatif tinggi} dan {jarak} m dari titik genangan {nama}.
```

Bila `data_cukup = false`: `"Belum ada laporan saluran di RW ini, skor sumbatan belum bisa dihitung."` Bila hujan dari simulasi, tambahkan `" (hujan simulasi demo)"`.

10. Antarmuka modul AI (`backend/src/ai/`, dikerjakan C)

Tabel 13 memberi integrasi AI dan validasinya ke peran C. Supaya B dan C tidak saling tunggu, batasnya dua fungsi ini. B memanggil, C mengisi.

```
// backend/src/ai/index.js
/**
 * @param {{ foto: Buffer, mimeType: string, jenisSaluran: 'terbuka'|'gorong_gorong'|'inlet_jalan' }} input
 * @returns {Promise<{ ok: true, sumber: 'model'|'demo', result: AiResult } | { ok: false, alasan: string }>}
 */
async function analyzeReport(input) {}

/**
 * @param {{ fotoSebelum: Buffer, fotoSesudah: Buffer, mimeType: string }} input
 * @returns {Promise<{ ok: true, sumber: 'model'|'demo', result: VerifyResult } | { ok: false, alasan: string
}>}
 */
async function verifyCleaning(input) {}

module.exports = { analyzeReport, verifyCleaning };
```

- `AiResult` = format Lampiran C. `VerifyResult` = format verifikasi di `docs/API.md`.
- Sebelum modul asli jadi, C menaruh **stub** yang langsung mengembalikan contoh Lampiran C. Dengan begitu B bisa menyelesaikan `POST /reports` di jam 1–3 tanpa menunggu.
- Model dipanggil lewat API model multimodal dengan structured output / JSON schema (3.3). Provider dan nama model dibaca dari `AI_API_KEY` dan `AI_MODEL`. Key **hanya** ada di backend, tidak pernah dikirim ke dasbor atau aplikasi.
- **Validasi keluaran** (wajib di modul, `RENCANA_FINAL.md` bagian 5):
  - Semua enum valid. `penyebab` subset dari `plastik`, `sedimen`, `daun_ranting`, `puing`.
  - `skor_sumbatan` cocok dengan kelas **[ASUMSI]**: bersih 0–0,15 · ringan 0,15–0,40 · sedang 0,40–0,70 · berat 0,70–1.

    - `tidak_terlihat` → `skor_sumbatan = null`, `saluran_terlihat = false`.
- **Batas waktu** 20 detik per panggilan **[ASUMSI]**, ulang 1 kali bila JSON tidak valid. Paling lama sekitar 40 detik, masih di bawah timeout mobile 60 detik.
- `DEMO_MODE=1` + gagal → hasil contoh bergiliran, `sumber: 'demo'` (FR-BE-19).
- Log per panggilan: durasi, `sumber`, dan alasan gagal. Jangan me-log isi key.

11. Notifikasi FCM

- Pakai `firebase-admin` dengan `FIREBASE_SERVICE_ACCOUNT_PATH`. File service account **tidak** di-commit (sudah di `.gitignore`).
- Payload tugas baru:

```
{
  "token": "<fcm_token petugas>",
  "notification": { "title": "Tugas baru", "body": "Bersihkan saluran terbuka di RW 05 (sumbatan berat)" },
  "data": { "type": "tugas_baru", "tugas_id": "12", "saluran_id": "7" }
}
```

- Semua nilai di `data` **wajib string**. Angka akan ditolak FCM.
- Token tidak valid (`messaging/registration-token-not-registered`) → kosongkan `pengguna.fcm_token`, respons `notifikasi: "gagal"`.

12. Kebutuhan non-fungsional

Kode	Kebutuhan
NFR-01 Kinerja	`POST /reports` dan `/verify` selesai ≤ 20 detik dalam kondisi normal (didominasi AI). `GET /rw/risk`, `/saluran`, `/priorities` ≤ 500 ms untuk data contoh
NFR-02 Ketahanan	Open-Meteo, FCM, atau AI gagal tidak boleh membuat endpoint lain error (FR-BE-06, 08, 14)
NFR-03 Keamanan	Rahasia hanya di `.env`. JWT_SECRET acak. Password bcrypt. Multer membatasi ukuran dan tipe. Error tanpa stack trace. `backend/uploads/` masuk `.gitignore` (kecuali foto seed di `db/seed/foto/`)
NFR-04 CORS	Izinkan origin dasbor (lokal `http://localhost:5173` dan domain deploy). Mobile tidak terkena CORS
NFR-05 Deploy	Satu URL HTTPS publik untuk API + `/uploads`. Database PostGIS di layanan cloud. Setelah HTTPS jalan, mobile bisa mematikan `usesCleartextTraffic`
NFR-06 Format	Waktu UTC ISO 8601, id angka, GeoJSON `[lng, lat]`, pesan error Bahasa Indonesia
NFR-07 Observabilitas	Log per request (method, path, status, durasi) + log AI (bagian 10)
NFR-08 Reproduksi	`npm run seed` deterministik. README berisi cara run: docker PostGIS → `npm i` → `npm run seed` → `npm run dev`
NFR-09 Kejujuran klaim (3.3)	Skor per RW, bukan per ruas jalan. `ai_sumber` dan `hujan_sumber` selalu dikirim supaya hasil demo/simulasi tidak tertukar dengan data asli

13. Uji terima (dasar Tabel 8 + `RENCANA_FINAL.md` bagian 10)

#	Skenario	Langkah	Hasil yang diharapkan	FR
1	Laporan normal	`curl -F foto=@berat.jpg -F lat=… -F lng=… -F captured_at=<now> -F jenis_saluran=terbuka …/reports`	201, `ai.kelas_sumbatan` terisi, skor RW naik	02, 05, 09
2	Saluran tertutup	Kirim foto gorong-gorong tertutup	`status: cek_manual`, tidak dihitung di S, muncul di `prioritas.cek_manual`	06, 12
3	AI mati	Isi `AI_API_KEY` salah, `DEMO_MODE=0`	201, `ai_sumber: gagal`, `status: cek_manual`	06
4	AI mati + mode demo	Sama, `DEMO_MODE=1`	201, `ai_sumber: demo`	19

#	Skenario	Langkah	Hasil yang diharapkan	FR
5	Laporan berdekatan	Kirim 2 laporan berjarak < 15 m	Laporan kedua `digabung: true`, `saluran_id` sama	04
6	Foto lama	`captured_at` 2 jam lalu	400	03
7	Tugas ganda	`POST /tasks` dua kali untuk saluran sama	Kedua kalinya 409	14
8	Petugas tanpa token FCM	`POST /tasks`	201, `notifikasi: tanpa_token`, tugas muncul di `GET /tasks?mine=1`	14, 15
9	Verifikasi berhasil	`POST /tasks/:id/verify` dengan foto bersih	Tugas `selesai`, skor RW turun (bagian 9.4), laporan awal `ditangani` + `bukti`	17, 07
10	Verifikasi belum bersih	Foto masih kotor	200, `sudah_bersih: false`, tugas `dikerjakan`	17
11	Open-Meteo putus	Matikan internet setelah cache ada	`hujan_sumber: cache`, endpoint tetap 200	08
12	Simulasi hujan	`PUT /demo/hujan {mm:80}` lalu `{mm:null}`	Sumber berganti `simulasi_demo` ↔ `open_meteo`	20
13	Lapor dari luar wilayah	Kirim dari koordinat rumah tim dengan `DEMO_SALURAN_ID` terisi	Laporan menempel ke saluran demo, `lokasi_demo: true`	21

14. Urutan kerja B (Tabel 14)

Jam	Pekerjaan B	Selesai bila
0–1	Kunci `API.md` (salin bagian 7), tulis `schema.sql` (bagian 8), docker PostGIS, Express kosong + `/health`	Tim setuju kontrak di Discord
1–5	Seed (FR-BE-18), `POST /reports` dengan stub AI dari C, FR-BE-04, `GET /rw/risk` dengan H dummy, `GET /saluran`	**Checkpoint jam 5** (bagian 4)
5–8	Open-Meteo + cache (FR-BE-08), `GET /priorities`, `GET /saluran/:id`, `GET /reports/:id`, login + middleware	**Checkpoint jam 8**
8–10	`GET /petugas`, `POST/GET /tasks` + FCM, `POST /devices`, `POST /tasks/:id/verify`, FR-BE-19/20/21	**Checkpoint jam 10**, lalu fitur dibekukan
10–11	Deploy API + DB + seed di cloud, jalankan uji terima bagian 13	URL HTTPS dibagikan ke A dan C
11–12	README cara run, catatan laporan pengembangan, daftar semua [ASUMSI] yang benar-benar dipakai	Berkas siap dikumpulkan

15. Keputusan yang perlu disepakati (default berlaku bila tidak ada keberatan)

Topik	Default usulan	Bagian
`POST /reports` tanpa login	Boleh, sebagai warga anonim	FR-BE-01
Batas umur foto	60 menit, toleransi 5 menit ke depan	FR-BE-03
Radius penggabungan saluran	15 m	FR-BE-04
Radius verifikasi	30 m dari saluran	FR-BE-17
Saluran bersih di daftar prioritas	Tidak ditampilkan	FR-BE-12
Simulasi hujan	Global di memori, hanya saat `DEMO_MODE=1`	FR-BE-20
Laporan dari luar wilayah saat demo	Ditempel ke `DEMO_SALURAN_ID`, disebut ke juri	FR-BE-21
Penyimpanan foto	Disk lokal `uploads/` disajikan statis (cukup untuk demo)	7

PRD & SRS · Dasbor Kelurahan RIVORA

Pemilik: C · AI dan dasbor
Versi: 1.0 · 8 November 2026 · **Status:** usulan, disepakati di Discord sebelum jam 1
Acuan: proposal 3.1–3.4, Tabel 3, 4, 8, 12, 13, 14, Lampiran B · `docs/RENCANA_FINAL.md` · `docs/backend/SRS.md` (kontrak data, bagian 7)

Penanda: P0 wajib jalan di checkpoint jam 10 · **P1** setelah semua P0 hijau · **P2** kalau sempat · **[ASUMSI]** belum ditetapkan proposal · **[DEMO]** alat bantu presentasi, wajib disebut ke juri.

Bagian I · PRD

1. Tujuan

Dasbor web dipakai kelurahan untuk menjawab pertanyaan utama RIVORA: **saluran mana yang perlu dibersihkan lebih dulu sebelum hujan turun** (3.1). Dasbor harus:

1. Menampilkan **peta risiko per RW** dan titik saluran bermasalah (Tabel 3 "Peta risiko").
2. Menampilkan **daftar prioritas** saluran saat hujan lebat diprediksi (Tabel 3).
3. Memungkinkan lurah **menugaskan petugas** dan memantau statusnya (Tabel 3 "Penugasan dan notifikasi").
4. Menunjukkan **bukti sebelum–sesudah** pembersihan (Tabel 3 "Verifikasi sebelum–sesudah").
5. Menjelaskan **kenapa sebuah RW berisiko**, supaya rekomendasinya bisa dipercaya (3.3, transparansi model).

2. Pengguna

Pengguna	Kebutuhan (Tabel 4)	Yang dilakukan di dasbor
Lurah / koordinator lapangan (utama)	Tahu kondisi saluran, prioritas kerja jelas, bukti kinerja	Lihat peta, baca alasan skor, tugaskan petugas, cek bukti
Kecamatan / dinas teknis	Data agregat untuk perencanaan	Lihat peta dan riwayat. Tampilan multi-kelurahan = tahap lanjut

Prototipe hanya menampilkan **satu kelurahan** dari data contoh **[ASUMSI]**.

3. Cakupan

Masuk: peta risiko RW, pin saluran, panel "Kenapa skor ini?", kartu prakiraan hujan, daftar prioritas, daftar cek manual, detail saluran, tugaskan petugas, daftar tugas, tampilan sebelum–sesudah, login lurah, simulasi hujan [DEMO].

Tidak masuk (Tabel 3 "tahap lanjut", 2.2): rekap & ekspor bulanan, mode kemarau, peringatan warga, integrasi LAPOR!/Command Center, tampilan multi-kelurahan, tampilan mobile.

4. Ukuran sukses (checkpoint `RENCANA_FINAL.md` bagian 8)

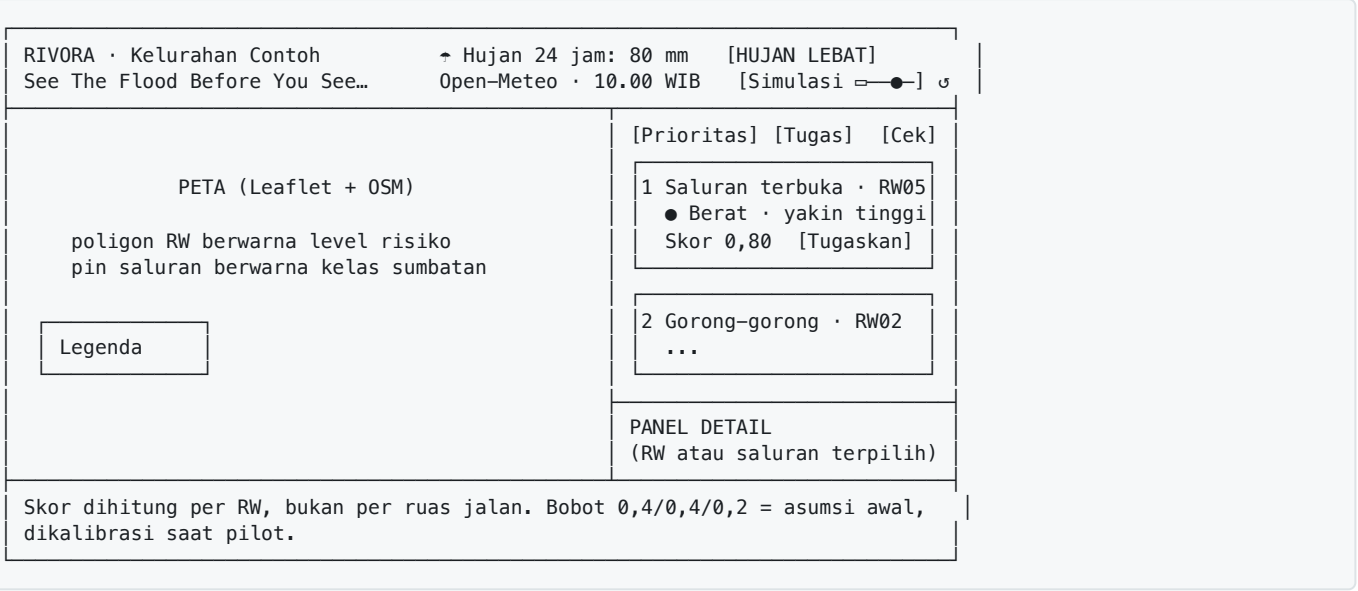
Jam	Bukti
5	Dasbor jalan dengan data tiruan (`VITE_USE_MOCK=1`): peta RW berwarna, pin saluran, legenda
8	Dasbor memakai API asli: peta risiko + daftar prioritas + prakiraan Open-Meteo + panel "Kenapa skor ini?"
10	Demo penuh: laporan dari HP muncul di peta ≤ 10 detik → Tugaskan → status tugas berubah → setelah verifikasi, foto sebelum–sesudah tampil dan warna RW turun

---

Bagian II · SRS

5. Tata letak

Satu halaman utama setelah login. Desktop lebar (layar presentasi/screen share), minimal 1280×720 **[ASUMSI]**.



Panel kanan sekitar 380 px, peta mengisi sisanya. Panel detail muncul di bawah daftar saat RW atau saluran diklik, dan bisa ditutup.

6. Kebutuhan fungsional

6.1 Akses

FR-DB-01 · Login lurah (P1) · Tabel 12 "Autentikasi"

- Form username + password → `POST /auth/login`. Hanya role `lurah` yang boleh masuk; role lain → pesan "Akun ini bukan akun kelurahan".
- Token disimpan di `localStorage` **[ASUMSI, cukup untuk prototipe]** dan dikirim sebagai `Authorization: Bearer`.
- Respons 401 di mana pun → hapus token, kembali ke halaman login.
- Sebelum login jadi, peta tetap bisa dikerjakan karena endpoint baca terbuka (`docs/backend/SRS.md` FR-BE-01).

6.2 Prediksi

FR-DB-02 · Kartu prakiraan hujan (P0) · Tabel 2 "Hujan"

- Sumber: `meta` dari `GET /rw/risk`.
- Tampilkan: total hujan 24 jam (mm, bilangan bulat), sumber, dan jam pengambilan dalam WIB.
- `hujan_lebat_diprediksi = true` → lencana merah **"Hujan lebat diprediksi"**.
- Teks sumber: `open_meteo` → "Open-Meteo" · `cache` → "Open-Meteo (data tersimpan)" · `tidak_tersedia` → "Prakiraan tidak tersedia" (abu-abu) · `simulasi_demo` → lihat FR-DB-14.

FR-DB-03 · Peta risiko per RW (P0) · Tabel 3 "Peta risiko", 3.2 langkah 2

- Leaflet + tile OpenStreetMap dengan atribusi "© OpenStreetMap contributors" (wajib menurut kebijakan OSM).
- Gambar poligon dari `GET /rw/risk` (GeoJSON). Warna mengikuti `level` (bagian 8). RW dengan `komponen.sumbatan.data_cukup = false` diberi garis putus-putus dan label "belum ada data".
- Hover → tooltip `RW 05 · Risiko tinggi · 0,80`. Klik → buka panel RW (FR-DB-06).
- Saat pertama dimuat, peta di-zoom pas ke seluruh poligon (`fitBounds`).
- Level selalu ditulis dalam teks di tooltip, panel, dan legenda. Jangan hanya mengandalkan warna.

FR-DB-04 · Pin saluran (P0) · Tabel 3 "titik saluran bermasalah"

- Dari `GET /saluran`. Pakai `CircleMarker` berwarna sesuai `kondisi.kelas` (bagian 8). Saluran `perlu_cek_manual` berwarna abu-abu.
- Saluran yang punya `tugas_aktif` diberi cincin luar biru supaya kelihatan sedang ditangani.
- Klik → buka panel saluran (FR-DB-09).

FR-DB-05 · Legenda (P0)

- Pojok kiri bawah peta: tiga level RW + "belum ada data", lima kelas saluran, dan penanda "sedang ditugaskan".

FR-DB-06 · Panel "Kenapa skor ini?" (P0) · 3.3 "transparansi model"

- Isi untuk RW terpilih:

```
RW 05 · RISIKO TINGGI · 0,80
Hujan       ▇▇▇▇▇▇▇▒▒  0,80 × 0,4 = 0,32   (80 mm / 24 jam)
Sumbatan    ▇▇▇▇▇▇▇▒▒  0,85 × 0,4 = 0,34   (1 saluran, 0 perlu cek manual)
Kerentanan  ▇▇▇▇▇▇▒▒▒  0,70 × 0,2 = 0,14   (lokasi rendah, 500 m dari Pasar Induk Gedebage)
"Risiko tinggi: prakiraan hujan 80 mm dalam 24 jam, 1 dari 1 saluran tersumbat sedang–berat, …"
```

- Angka diambil langsung dari `properties.komponen` dan `properties.alasan`. **Dasbor tidak menghitung ulang skor**, supaya angkanya pasti sama dengan backend.
- Di bawahnya: daftar saluran di RW itu (klik → panel saluran).

6.3 Bertindak

FR-DB-07 · Daftar prioritas sebelum hujan (P0) · 3.2 langkah 3, Tabel 3

- Dari `GET /priorities`, urutan sesuai `peringkat` dari backend.
- Tiap kartu: peringkat, jenis saluran + RW, lencana kelas, keyakinan AI (**selalu tampil**, Tabel 8), skor prioritas, umur laporan ("2 jam lalu"), dan tombol **Tugaskan**. Bila sudah punya `tugas_aktif`, tombol diganti status tugas + nama petugas.
- Header daftar:
  - `hujan_lebat_diprediksi = true` → "Bersihkan sebelum hujan: N saluran"
  - `false` → "Tidak ada hujan lebat 24 jam ke depan. Urutan berdasarkan kondisi saluran." (daftar tetap tampil)
- Klik kartu → peta terbang ke saluran itu dan panel saluran terbuka.

FR-DB-08 · Daftar perlu cek manual (P1) · 3.2, Tabel 8 "kelas tidak terlihat memicu pengecekan manual"

- Tab **Cek**: isi `cek_manual` dari `GET /priorities`. Foto mini, RW, catatan AI, waktu. Tombol Tugaskan tetap ada (petugas diminta mengecek langsung).

FR-DB-09 · Detail saluran (P0)

- Dari `GET /saluran/:id`:
  - Foto terbaru (klik → perbesar).
  - Lencana kelas, skor sumbatan, **keyakinan AI**, penyebab, catatan AI, waktu foto.
  - Keyakinan `rendah` → keterangan "Perlu dikonfirmasi petugas".
  - Riwayat laporan (terbaru dulu), yang berasal dari verifikasi ditandai "Bukti pembersihan".
  - Status tugas + tombol Tugaskan.
- Laporan dengan `ai_sumber = demo` atau `lokasi_demo = true` diberi label kecil "contoh demo" **[DEMO]**, supaya jujur saat presentasi.

FR-DB-10 · Tugaskan petugas (P0) · Tabel 3 "Penugasan dan notifikasi"

- Dialog: ringkasan saluran, pilihan petugas dari `GET /petugas` (nama + jumlah tugas aktif), catatan opsional, tombol **Kirim tugas**.
- `POST /tasks` → toast sesuai `notifikasi`:
  - `terkirim` → "Tugas dikirim ke Petugas 1"
  - `tanpa_token` / `gagal` → "Tugas dibuat, tapi notifikasi tidak terkirim. Petugas tetap melihatnya di daftar tugas."
- 409 → "Saluran ini sudah punya tugas aktif". Setelah sukses, semua data dimuat ulang (FR-DB-13).
- Tombol dimatikan selama request berjalan supaya tidak terkirim dua kali.

FR-DB-11 · Daftar tugas (P0)

- Tab **Tugas**: `GET /tasks?kelurahan_id=`, dikelompokkan **Baru** → **Dikerjakan** → **Selesai**.
- Tiap kartu: saluran + RW, petugas, waktu dibuat, status. Tugas selesai menampilkan miniatur sebelum–sesudah.

6.4 Verifikasi

FR-DB-12 · Tampilan sebelum–sesudah (P0) · 3.2 langkah 4, Tabel 3

- Di panel saluran dan kartu tugas selesai: dua foto berdampingan, berlabel **Sebelum** (kelas + skor) dan **Sesudah** (kelas + skor), plus `catatan` dan keyakinan dari verifikasi AI, plus waktu selesai.
- **P2:** slider geser untuk membandingkan dua foto.

6.5 Pembaruan data dan demo

FR-DB-13 · Muat ulang otomatis (P0)

- Setiap **10 detik [ASUMSI]**, muat ulang `GET /rw/risk`, `GET /saluran`, `GET /priorities`, `GET /tasks`. Langsung muat ulang juga setelah aksi (tugaskan, ubah simulasi).
- Inilah yang membuat laporan dari HP muncul di dasbor saat demo tanpa harus refresh.
- Saat dimuat ulang, posisi dan zoom peta serta pilihan panel **tidak** boleh berubah.
- Request gagal → banner kuning "Tidak terhubung ke server. Menampilkan data terakhir (10.32 WIB)". Data lama tetap tampil.

FR-DB-14 · Simulasi hujan (P1) [DEMO] · `RENCANA_FINAL.md` bagian 12

- Hanya tampil bila `GET /health` → `demo_mode: true`.
- Slider 0–100 mm. Nilai dikirim lewat `PUT /demo/hujan { mm }` **saat slider dilepas**, bukan setiap kali digeser. Tombol ↺ mengirim `{ mm: null }`.
- Selama `hujan_sumber = simulasi_demo`: banner oranye di atas peta **"SIMULASI DEMO: hujan diatur 80 mm, bukan prakiraan asli"**. Tidak bisa disembunyikan.

FR-DB-15 · Catatan batas klaim (P0) · 3.3

- Footer tetap: "Skor dihitung per RW, bukan per ruas jalan. Bobot 0,4/0,4/0,2 adalah asumsi awal dan akan dikalibrasi saat pilot."

FR-DB-16 · Sorot perubahan level (P2)

- Bila level sebuah RW berubah antara dua kali muat ulang, poligonnya berkedip 3 detik dan muncul toast "RW 05: Risiko tinggi → Waspada". Membantu juri menangkap momen verifikasi.

FR-DB-17 · Mode data tiruan (P0, untuk pengembangan)

- `VITE_USE_MOCK=1` → klien API membaca `src/mocks/*.json` yang bentuknya **persis** contoh di `docs/backend/SRS.md` bagian 7. C bisa membangun seluruh UI di jam 1–5 tanpa menunggu backend.
- Mock `POST /tasks` cukup mengubah state di memori supaya alur tugaskan bisa dicoba.

7. Data dari backend

Kebutuhan	Endpoint (`docs/backend/SRS.md`)	Kapan dipanggil
Login	`POST /auth/login` (7.1)	Form login
Poligon RW + skor + kartu hujan	`GET /rw/risk?kelurahan_id=` (7.4)	Awal + tiap 10 detik
Pin saluran	`GET /saluran?kelurahan_id=` (7.5)	Awal + tiap 10 detik
Detail saluran	`GET /saluran/:id` (7.6)	Saat pin/kartu diklik, dan saat muat ulang bila panel terbuka
Prioritas + cek manual	`GET /priorities?kelurahan_id=` (7.7)	Awal + tiap 10 detik
Pilihan petugas	`GET /petugas` (7.8)	Saat dialog Tugaskan dibuka
Buat / daftar tugas	`POST /tasks`, `GET /tasks?kelurahan_id=` (7.9)	Tugaskan; awal + tiap 10 detik

Kebutuhan	Endpoint (`docs/backend/SRS.md`)	Kapan dipanggil
Simulasi hujan	`PUT /demo/hujan` (7.11)	Slider dilepas / tombol ↺
Mode demo aktif?	`GET /health` (7.11)	Sekali saat awal

Konfigurasi `dashboard/.env` :

```
VITE_API_URL=http://localhost:3000
VITE_KELURAHAN_ID=1
VITE_USE_MOCK=0
```

8. Spesifikasi visual

Warna kelas sumbatan, sama dengan aplikasi mobile (`mobile/lib/widgets/sumbatan_style.dart`) supaya app dan dasbor terlihat satu sistem:

Kelas	Label	Warna
`bersih`	Bersih	`#43A047`
`ringan`	Sumbatan ringan	`#9E9D24`
`sedang`	Sumbatan sedang	`#F57C00`
`berat`	Sumbatan berat	`#D32F2F`
`tidak_terlihat`	Tidak terlihat	`#607D8B`

Warna level RW:

Level	Label	Isi poligon	Garis
`aman`	Aman	`#43A047` , opacity 0,30	`#43A047` , 2 px
`waspada`	Waspada	`#F57C00` , opacity 0,35	`#F57C00` , 2 px
`risiko_tinggi`	Risiko tinggi	`#D32F2F` , opacity 0,40	`#D32F2F` , 2 px
belum ada data	Belum ada data	`#9E9E9E` , opacity 0,15	`#9E9E9E` , 2 px putus-putus

Format dan bahasa:

- Semua teks Bahasa Indonesia. Label enum dari tabel di atas, jangan tampilkan `risiko_tinggi` mentah.
- Desimal pakai koma (`0,80`) lewat `Intl.NumberFormat('id-ID', { minimumFractionDigits: 2, maximumFractionDigits: 2 })` .
- Waktu dalam WIB: `Intl.DateTimeFormat('id-ID', { timeZone: 'Asia/Jakarta', hour: '2-digit', minute: '2-digit' })` + " WIB". Umur laporan relatif ("5 menit lalu").
- Jenis saluran: `terbuka` → Saluran terbuka · `gorong_gorong` → Gorong-gorong · `inlet_jalan` → Inlet jalan.
- Penyebab: `plastik` → Plastik · `sedimen` → Sedimen · `daun_ranting` → Daun & ranting · `puing` → Puing.
- Keyakinan: Rendah / Sedang / Tinggi, selalu dengan kata "Keyakinan AI".

9. Kebutuhan non-fungsional

Kode	Kebutuhan
NFR-01 Browser	Chrome versi terbaru di desktop. Layout tidak pecah di 1280×720 dan tetap terbaca di 1024 px
NFR-02 Kinerja	Muat awal ≤ 2 detik di jaringan biasa. Muat ulang 10 detik tidak membuat peta berkedip atau kehilangan posisi
NFR-03 Ketahanan	Backend mati → banner + data terakhir, tidak layar putih. Foto gagal dimuat → kotak abu-abu "Foto tidak tersedia"
NFR-04 Aksesibilitas dasar	Level dan kelas selalu ditulis dalam teks, bukan hanya warna. Kontras teks memadai. Tombol bisa dipakai dengan keyboard
NFR-05 Keamanan	Tidak ada API key apa pun di kode dasbor (AI dipanggil backend). `.env` tidak di-commit

Kode	Kebutuhan
NFR-06 Kejujuran klaim	Label simulasi dan "contoh demo" selalu tampil saat berlaku (FR-DB-09, 14). Footer batas klaim (FR-DB-15)
NFR-07 Deploy	Build statis (`npm run build`) di-hosting di layanan cloud. `VITE_API_URL` diarahkan ke URL HTTPS backend

10. Teknis

Stack: React + Vite + `leaflet` + `react-leaflet` (Tabel 12, `RENCANA_FINAL.md` bagian 3). Library UI tidak wajib, cukup CSS biasa.

Struktur yang disarankan:

```
dashboard/src/
├── main.jsx
├── App.jsx                  # cek token → Login atau Dashboard
├── api/client.js            # fetch + token + base URL + switch mock
├── mocks/                   # rw-risk.json, saluran.json, priorities.json, tasks.json, petugas.json, saluran-
7.json
├── hooks/usePolling.js      # muat ulang tiap 10 detik
├── format.js                # label enum, angka, waktu WIB
├── pages/
│   ├── Login.jsx
│   └── Dashboard.jsx
└── components/
    ├── RainCard.jsx  DemoRainSlider.jsx
    ├── RiskMap.jsx   RwLayer.jsx  SaluranLayer.jsx  Legend.jsx
    ├── PriorityList.jsx  ManualCheckList.jsx  TaskList.jsx
    ├── RwDetail.jsx  SaluranDetail.jsx  BeforeAfter.jsx
    └── AssignDialog.jsx  Toast.jsx
```

Jebakan Leaflet yang sering makan waktu:

- Lupa `import 'leaflet/dist/leaflet.css'` → tile acak-acakan.
- `MapContainer` wajib punya tinggi eksplisit (mis. `height: 100%` dengan parent setinggi layar), kalau tidak peta tidak terlihat.
- `<GeoJSON>` di react-leaflet **tidak menggambar ulang** saat `data` berubah. Beri `key={meta.dihitung_at}` supaya dibuat ulang tiap data baru.
- Ikon `Marker` bawaan rusak di Vite (path gambar). Pakai `CircleMarker` saja.
- GeoJSON memakai `[lng, lat]`, sedangkan Leaflet `LatLng` memakai `[lat, lng]`. Field `lat` / `lng` di JSON biasa sudah terpisah, jadi aman.
- `fitBounds` hanya sekali di awal. Jangan dipanggil setiap muat ulang, atau peta akan melompat terus.

11. Uji terima

#	Skenario	Hasil yang diharapkan	FR
1	Buka dasbor dengan data contoh	Poligon RW berwarna campuran, pin saluran, legenda, kartu hujan	02–05
2	Klik RW	Panel menampilkan 3 komponen, angka sama dengan respons API, kalimat alasan	06
3	RW tanpa laporan	Garis putus-putus + "belum ada data", bukan hijau	03
4	Kirim laporan "berat" dari HP	Dalam ≤ 10 detik pin baru muncul, warna RW dan urutan prioritas berubah tanpa refresh	04, 07, 13
5	Laporan "tidak terlihat"	Muncul di tab Cek, pin abu-abu	04, 08
6	Tugaskan petugas	Toast sesuai `notifikasi`, kartu prioritas menampilkan status tugas, tab Tugas bertambah	10, 11
7	Tugaskan saluran yang sama lagi	Pesan "sudah punya tugas aktif"	10

#	Skenario	Hasil yang diharapkan	FR
8	Petugas verifikasi dari HP	Tugas pindah ke Selesai, sebelum–sesudah tampil, skor dan warna RW turun	11, 12, 13
9	Geser simulasi ke 80 mm, lalu ↺	Banner simulasi muncul / hilang, lencana hujan lebat ikut berubah	02, 14
10	Matikan backend	Banner "Tidak terhubung", data terakhir tetap tampil	13
11	Token kedaluwarsa	Kembali ke login	01

12. Urutan kerja C untuk dasbor (Tabel 14)

C juga memegang modul AI (`docs/backend/SRS.md` bagian 10). Jam AI ditulis di kolom terpisah supaya beban kerjanya kelihatan.

Jam	AI (`backend/src/ai/`)	Dasbor	Selesai bila
0–1	Taruh stub `analyzeReport` / `verifyCleaning` , kumpulkan 10–15 foto uji	`npm create vite` , pasang Leaflet, isi `src/mocks/` dari contoh bagian 7 SRS backend	Stub dan mock di-commit
1–5	Prompt + structured output + validasi, uji ke semua foto	Peta + RW layer + pin + legenda dengan mock	**Checkpoint jam 5** (bagian 4)
5–8	Perbaiki prompt dari hasil uji	Pindah ke API asli, kartu hujan, panel "Kenapa skor ini?", daftar prioritas, detail saluran, polling	**Checkpoint jam 8**
8–10	Prompt verifikasi sebelum–sesudah	Login, dialog Tugaskan, tab Tugas, sebelum–sesudah, tab Cek, simulasi hujan	**Checkpoint jam 10**, lalu fitur dibekukan
10–11	–	Deploy dasbor, jalankan uji terima bagian 11	URL dasbor dibagikan
11–12	–	Jalankan alur untuk rekaman video, ambil screenshot untuk PPT dan laporan	Berkas siap

Bila waktu mepet, yang dipotong lebih dulu: FR-DB-16 → slider sebelum–sesudah (P2) → FR-DB-08 → FR-DB-01 (dasbor tanpa login, tugaskan memakai token lurah yang di-hardcode di `.env` lokal, **jangan di-commit**).

13. Keputusan yang perlu disepakati (default berlaku bila tidak ada keberatan)

Topik	Default usulan
Interval muat ulang	10 detik
Jumlah kelurahan	Satu, dari `VITE_KELURAHAN_ID`
Ukuran layar target	Desktop ≥ 1280×720, tidak ada tampilan mobile
Penyimpanan token	`localStorage`
Saluran bersih di daftar prioritas	Tidak tampil (mengikuti backend FR-BE-12)

PRD & SRS · Aplikasi Pelapor RIVORA (Mobile)

Pemilik: A · Mobile developer · **Branch:** `feature/mobile`
Versi: 1.0 · 8 November 2026 · **Status:** usulan, disepakati di Discord
Acuan: proposal 3.1, 3.2, 6.2, Tabel 3, 8, 12, 13, 14, Lampiran C · `docs/backend/SRS.md` (kontrak data, bagian 7) · `docs/mobile/01-alur-lapor.md`

Penanda: P0 wajib jalan di checkpoint jam 10 · **P1** setelah semua P0 hijau · **P2** kalau sempat · **[ASUMSI]** belum ditetapkan proposal · **[DEMO]** alat bantu presentasi, wajib disebut ke juri · ✅ sudah jadi · ⬜ belum.

Bagian I · PRD

1. Tujuan

Aplikasi Android untuk petugas kebersihan, siskamling, dan warga (3.1). Aplikasi ini memegang dua ujung siklus RIVORA (3.2):

1. **Lapor:** memotret saluran langsung dari kamera, GPS dan waktu tercatat otomatis, lalu menampilkan hasil analisis AI.
2. **Verifikasi:** petugas menerima tugas lewat notifikasi, membersihkan saluran, lalu memotret ulang sebagai bukti.

2. Pengguna

Pengguna	Login	Yang dilakukan
Warga	Tidak wajib **[ASUMSI]**	Lapor saluran, lihat status laporannya
Petugas kebersihan / siskamling	Wajib (akun dari kelurahan)	Lapor, terima tugas, kirim foto sesudah

Proposal menjadikan petugas pelapor utama dan warga pelapor tambahan (Tabel 8, mitigasi partisipasi rendah).

3. Cakupan

Masuk: lapor foto + GPS, hasil AI, status laporan, login petugas, daftar tugas, notifikasi tugas (FCM), foto sesudah untuk verifikasi.

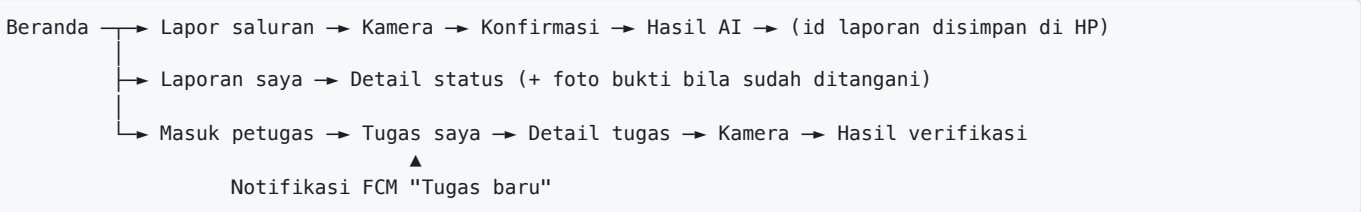
Tidak masuk (Tabel 3 "tahap lanjut"): peringatan warga di wilayah berisiko, mode kemarau, versi iOS.

4. Ukuran sukses

Jam	Bukti
5	Foto dari HP asli → backend asli → hasil AI asli tampil di layar hasil
8	Laporan saya menampilkan status dari backend. Petugas bisa login dan melihat daftar tugas
10	Lurah menugaskan di dasbor → notifikasi masuk di HP petugas → foto sesudah → "terverifikasi bersih"
10–11	APK release mengarah ke backend HTTPS, terpasang di minimal 2 HP Android (6.2)

Bagian II · SRS

5. Alur layar



6. Kebutuhan fungsional

6.1 Sudah jadi di tahap 01 (`docs/mobile/01-alur-lapor.md`)

Kode	Kebutuhan	Prioritas	Status
FR-MB-01	Foto langsung dari kamera in-app, **tanpa galeri** (3.2)	P0	✅
FR-MB-02	GPS dan waktu tercatat otomatis, lokasi palsu ditolak (3.2, Tabel 8)	P0	✅
FR-MB-03	Pilih jenis saluran: terbuka, gorong-gorong, inlet jalan (3.2)	P0	✅
FR-MB-04	Kirim `POST /reports`, tampilkan kelas, skor, penyebab, **keyakinan AI selalu tampil**, banner "Perlu dicek manual" (Tabel 8, Lampiran C)	P0	✅ mode contoh · ⬜ uji dengan backend asli
FR-MB-05	Mode contoh (`MockApiClient`) bila `API_URL` kosong	P0	✅

6.2 Belum dikerjakan

FR-MB-06 · Laporan saya (P1) · Tabel 8 "status sudah ditangani beserta foto dikirim kembali ke pelapor"

- Setiap laporan yang berhasil dikirim disimpan di HP (`shared_preferences`): id, waktu, jenis saluran, kelas. Warga tanpa login tetap bisa melacak laporannya.
- Layar daftar → ketuk → `GET /reports/:id` → tampilkan label status (`dikirim` / `ditugaskan` / `ditangani` / `cek_manual`).
- Status `ditangani` → tampilkan foto bukti (`bukti.foto_sesudah_url`), kelas sesudah, dan waktu selesai.
- Tarik ke bawah untuk muat ulang.

FR-MB-07 · Masuk petugas (P0) · Tabel 12 "Autentikasi"

- `POST /auth/login`. Di layar ini hanya role `petugas` yang diterima **[ASUMSI]**. Role lain → "Akun ini bukan akun petugas".
- Token disimpan di `shared_preferences`, dikirim sebagai `Authorization: Bearer` ke endpoint petugas. `POST /reports` juga mengirim token bila ada, supaya pelapor tercatat.
- Tombol Keluar menghapus token. Respons 401 → kembali ke layar login.

FR-MB-08 · Tugas saya (P0) · 3.2 langkah 3

- `GET /tasks?mine=1`. Tugas `baru` dan `dikerjakan` di atas, `selesai` di bawah.
- Kartu: jenis saluran + RW, foto sebelum + kelas sebelum, catatan dari lurah, waktu ditugaskan.
- Detail tugas: tombol **Foto sesudah** (FR-MB-10). **P1:** tombol **Buka di peta** (`url_launcher` ke Google Maps dengan `lat,lng`).
- Daftar dimuat ulang saat layar dibuka, saat app kembali ke depan, dan saat ditarik ke bawah. Ini cadangan bila notifikasi tidak sampai (`docs/backend/SRS.md` FR-BE-15).

FR-MB-09 · Notifikasi tugas (P0) · Tabel 3 "Penugasan dan notifikasi", Tabel 12 "Firebase Cloud Messaging"

- Paket `firebase_core` + `firebase_messaging`. `google-services.json` dari proyek Firebase yang sama dengan service account backend.
- Setelah login: minta izin notifikasi (Android 13+), ambil token FCM, kirim `POST /devices`. Saat `onTokenRefresh` → kirim ulang.

- Payload dari backend: `data.type = "tugas_baru"`, `data.tugas_id`, `data.saluran_id` (semua string).
- App di belakang / tertutup: notifikasi tampil otomatis dari blok `notification`. Ketuk → buka detail tugas (`getInitialMessage`, `onMessageOpenedApp`).
- App di depan: Android tidak menampilkan notifikasi FCM otomatis → tampilkan banner/SnackBar "Tugas baru masuk" dan muat ulang daftar **[ASUMSI: tanpa `flutter_local_notifications`, supaya cepat]**.

FR-MB-10 · Foto sesudah / verifikasi (P0) · 3.2 langkah 4, Tabel 3

- Pakai ulang layar kamera tahap 01: tanpa galeri, GPS + waktu otomatis, tolak lokasi palsu.
- `POST /tasks/:id/verify` multipart: `foto`, `lat`, `lng`, `captured_at` (UTC), dengan token petugas.
- Hasil:
  - `verifikasi.sudah_bersih = true` → "Saluran terverifikasi bersih", foto sebelum dan sesudah berdampingan, kelas sesudah, keyakinan AI.
  - `sudah_bersih = false` → tampilkan catatan AI + tombol **Foto ulang**.
  - `verifikasi = null` → tampilkan `pesan` dari backend + tombol **Foto ulang**.
- Backend menolak bila posisi lebih dari 30 m dari saluran (`docs/backend/SRS.md` FR-BE-17). Pesan error ditampilkan apa adanya.

FR-MB-11 · Tandai dikerjakan (P2)

- Tombol **Mulai kerjakan** → `PATCH /tasks/:id { "status": "dikerjakan" }`.

6.3 Label demo [DEMO]

- Bila respons berisi `ai_sumber = "demo"`, layar hasil menampilkan label kecil "Hasil contoh (mode demo)".
- Bila `lokasi_demo = true`, tampilkan "Lokasi dipetakan ke saluran contoh". Keduanya supaya jujur di depan juri (`docs/backend/SRS.md` FR-BE-19, FR-BE-21).

7. Endpoint yang dipakai

Fitur	Endpoint (`docs/backend/SRS.md` bagian 7)	Token
Lapor	`POST /reports` (7.2)	Opsional
Laporan saya	`GET /reports/:id` (7.3)	Opsional
Masuk petugas	`POST /auth/login` (7.1)	–
Simpan token FCM	`POST /devices` (7.11)	Petugas
Tugas saya	`GET /tasks?mine=1` (7.9)	Petugas
Foto sesudah	`POST /tasks/:id/verify` (7.10)	Petugas
Tandai dikerjakan (P2)	`PATCH /tasks/:id` (7.11)	Petugas

8. Kebutuhan non-fungsional

Kode	Kebutuhan
NFR-01 Perangkat	Android. Diuji di minimal 2 HP Android asli (6.2)
NFR-02 Jaringan	Timeout unggah 60 detik (analisis AI bisa sampai sekitar 40 detik). Pesan error Bahasa Indonesia, pesan dari backend ditampilkan apa adanya
NFR-03 Anti laporan palsu	Tidak ada akses galeri di seluruh app, termasuk verifikasi. Lokasi palsu ditolak kecuali build emulator (`ALLOW_MOCK_LOCATION`)
NFR-04 Izin	Kamera, lokasi, dan notifikasi diminta dengan penjelasan singkat. GPS mati → tombol ke Pengaturan
NFR-05 Build demo	`flutter build apk --release --dart-define=API_URL=https://<backend>`. Setelah backend HTTPS, `usesCleartextTraffic` dimatikan
NFR-06 Rahasia	`google-services.json` dan keystore tidak di-commit (sudah di `.gitignore`)
NFR-07 Konsistensi	Warna kelas sumbatan sama dengan dasbor (`docs/dashboard/SRS.md` bagian 8)

9. Uji terima

#	Skenario	Hasil yang diharapkan	FR
1	Lapor saluran berat ke backend asli	Hasil AI asli tampil, pin muncul di dasbor ≤ 10 detik	04
2	Lapor saluran tertutup	Banner "Perlu dicek manual", status `cek_manual`	04
3	Buka Laporan saya setelah lapor	Laporan tadi ada, status sesuai backend	06
4	Laporan sudah dibersihkan petugas	Status "Sudah ditangani" + foto bukti	06
5	Login dengan akun warga di layar petugas	Ditolak dengan pesan	07
6	Lurah menugaskan, app di belakang	Notifikasi muncul, ketuk → detail tugas	09
7	Lurah menugaskan, app di depan	Banner "Tugas baru masuk", daftar bertambah	09
8	Notifikasi tidak sampai	Tugas tetap muncul setelah tarik ke bawah	08
9	Foto sesudah saluran bersih	"Terverifikasi bersih", sebelum–sesudah tampil, status di dasbor jadi Selesai	10
10	Foto sesudah masih kotor	Catatan AI + tombol Foto ulang	10
11	Backend mati saat kirim	Pesan "Tidak bisa terhubung ke server", foto tidak hilang, bisa kirim ulang	04

10. Urutan kerja A (Tabel 14)

Jam	Pekerjaan A	Selesai bila
0–1	Setup Flutter + HP Android	✅
1–5	Alur lapor (FR-MB-01–05), lalu uji ke backend asli begitu `POST /reports` B jalan	✅ mode contoh · **Checkpoint jam 5**
5–8	Laporan saya (FR-MB-06), login petugas (FR-MB-07), kerangka Tugas saya (FR-MB-08)	**Checkpoint jam 8**
8–10	FCM (FR-MB-09), foto sesudah (FR-MB-10), label demo	**Checkpoint jam 10**, lalu fitur dibekukan
10–11	Build APK release, pasang di 2 HP, jalankan uji terima bagian 9	APK siap
11–12	Rekam video demo (wajib dikumpulkan)	Video jadi

11. Yang dibutuhkan dari anggota lain

Dari	Kebutuhan	Paling lambat
B	`POST /reports` jalan di laptop/cloud + alamatnya	Jam 5
B	Akun `petugas1` dari data contoh + password demo (lewat Discord, bukan di repo)	Jam 6
B	Proyek Firebase: `google-services.json` untuk A, service account untuk backend **[ASUMSI: B yang membuat]**	Jam 8
B	URL backend HTTPS hasil deploy	Jam 10
C	Konfirmasi warna kelas di dasbor sama dengan app	Jam 8